



JADAVPUR UNIVERSITY

Name: Diptarko Bhattacharjee

Roll: 002410501013

Class: BCSE - UG 3 - Sem 1 - A1

Subject: Computer Networks Lab Assignment 1 Report

Contents

1	Introduction	2
2	Inter-Process Communication (IPC)	2
2.1	TCP Socket Architecture and Stream Boundaries	2
3	System Architecture	3
3.1	Frame Layout	3
3.2	Flowchart and Data Flow	4
4	Types of Errors and Encoding Schemes	4
4.1	Encoding Schemes	4
4.2	Error Types and Deterministic Injection	5
5	Project File Structure	6
6	Core Modules & Code Implementation	6
6.1	frame.hpp and frame.cpp - Protocol Structure	6
6.1.1	Key Code Snippets: Data Structures and Serialization	6
6.1.2	Detailed Module Analysis	7
6.2	checksum.cpp - Internet Checksum	8
6.2.1	Key Code Snippet: Checksum Computation	8
6.2.2	Detailed Module Analysis	8
6.3	crc.cpp - Cyclic Redundancy Check	9
6.3.1	Key Code Snippet: Modulo-2 Polynomial Division	9
6.3.2	Detailed Module Analysis	9
6.4	error.cpp - Error Injection Logic	10
6.4.1	Key Code Snippets: Bit Flipping and Targeted Attacks	10
6.4.2	Detailed Module Analysis	11
6.5	utils.cpp - Network and Framework Utilities	12
6.5.1	Key Code Snippets: Socket Flow and Frame Building	12
6.5.2	Detailed Module Analysis	13
6.6	sender.cpp - The Client Application	13
6.6.1	Key Code Snippet: Core Transmission Loop	13
6.6.2	Detailed Module Analysis	13
6.7	receiver.cpp - The Server Daemon	14
6.7.1	Key Code Snippet: Core Verification Loop	14
6.7.2	Detailed Module Analysis	14
7	Experimental Results	15
7.1	Performance Metrics	15
7.2	Detailed Dry Run & Blind Spot Analysis	15
7.2.1	Frames 1-5: Standard Network Interference	17
7.2.2	Frame 6 Analysis: Exploiting the Checksum Blind Spot	18
7.2.3	Frame 7 Analysis: Exploiting the CRC Blind Spot	18
8	Discussion	18
9	Conclusion	19

Assignment 1: Design and Implement an Error Detection Module using Checksum and Cyclic Redundancy Check (CRC)

1 Introduction

In physical networks, data is inherently susceptible to noise, attenuation, and electromagnetic interference, which can flip bits during transmission over long distances or noisy mediums. To guarantee data integrity and ensure that corrupted frames are not processed as valid data by higher layers, the Data Link Layer employs robust error detection algorithms. These algorithms typically append redundant verification bits—often referred to as a Frame Check Sequence (FCS)—to the end of a payload before it is transmitted across the physical medium. Upon receiving the frame, the receiver re-calculates the verification sequence and compares it to the FCS appended by the sender.

The primary objective of this assignment is to design, implement, and rigorously test two classical error detection mechanisms: the **16-bit Internet Checksum** and the **Cyclic Redundancy Check (CRC)** (specifically CRC-8, CRC-10, CRC-16, and CRC-32). Rather than merely simulating this mathematically in a single, disconnected script, we built a fully functional client-server architecture using TCP Socket Programming in C++. This allows us to observe the mathematical blind spots of these algorithms in a realistic, distributed network environment, tracking the performance overhead and exact failure conditions of each scheme.

2 Inter-Process Communication (IPC)

To simulate a true network link where packets are serialized and deserialized across a physical boundary, our sender and receiver communicate using POSIX TCP Sockets.

2.1 TCP Socket Architecture and Stream Boundaries

Transmission Control Protocol (TCP) ensures a reliable, stream-oriented connection. However, since TCP is a stream protocol (meaning it does not inherently preserve application-layer message boundaries and can fragment packets), our application layer must enforce strict boundaries to prevent data corruption.

We achieve this by defining a rigid, fixed-size **128-byte frame**. The Sender exclusively pushes exactly 128 bytes to the socket per transmission. We implemented a custom ‘send_all’ and ‘recv_all’ loop in our socket utility module. This ensures that the Receiver explicitly waits, continually reading from the socket buffer, until exactly 128 bytes are pulled before it attempts any FCS verification. After processing the frame, the receiver

sends back a 5-byte Acknowledgement packet (either an “ACK” or “NACK”) indicating the integrity verdict.

3 System Architecture

The system relies on a tightly coupled sender-receiver loop. The sender chunks the input file, computes the FCS using the chosen scheme, maliciously injects targeted deterministic errors, and transmits the frame. The receiver strictly parses the incoming 128 bytes, separates the payload from the FCS, recomputes the FCS over the payload, and validates it.

3.1 Frame Layout

We designed a strict 128-byte frame layout heavily inspired by the actual IEEE 802.3 Ethernet frame standard. The byte-by-byte breakdown of each individual frame is rigidly defined as follows:

1. **6 bytes (Offset 0):** Destination MAC Address. Specifies the hardware address of the receiver.
2. **6 bytes (Offset 6):** Source MAC Address. Specifies the hardware address of the sender.
3. **2 bytes (Offset 12):** Valid payload length. Since the final chunk of a file might be less than 110 bytes, this 16-bit integer informs the receiver exactly how many bytes are valid application data versus zero-padded whitespace.
4. **110 bytes (Offset 14):** Payload data. The actual application-layer data being transmitted.
5. **4 bytes (Offset 124):** Frame Check Sequence (FCS). The redundant verification bits calculated by the Checksum or CRC logic. Though some CRCs are 8 or 16 bits, we allocate a full 4 bytes (32 bits) to accommodate CRC-32, right-justifying smaller schemes and padding with zeros.

Offset	Size (Bytes)	Field Description
0	6	Destination MAC Address
6	6	Source MAC Address
12	2	Valid Payload Length (≤ 110)
14	110	Payload Data (Zero-padded if short)
124	4	Frame Check Sequence (FCS)

Table 1: Custom 128-Byte Frame Structure

3.2 Flowchart and Data Flow

To visualize the interactions between these core modules, the following diagram illustrates the complete data flow. The ACK/NACK packet is securely routed back to the Sender to dictate the status of the transmission.

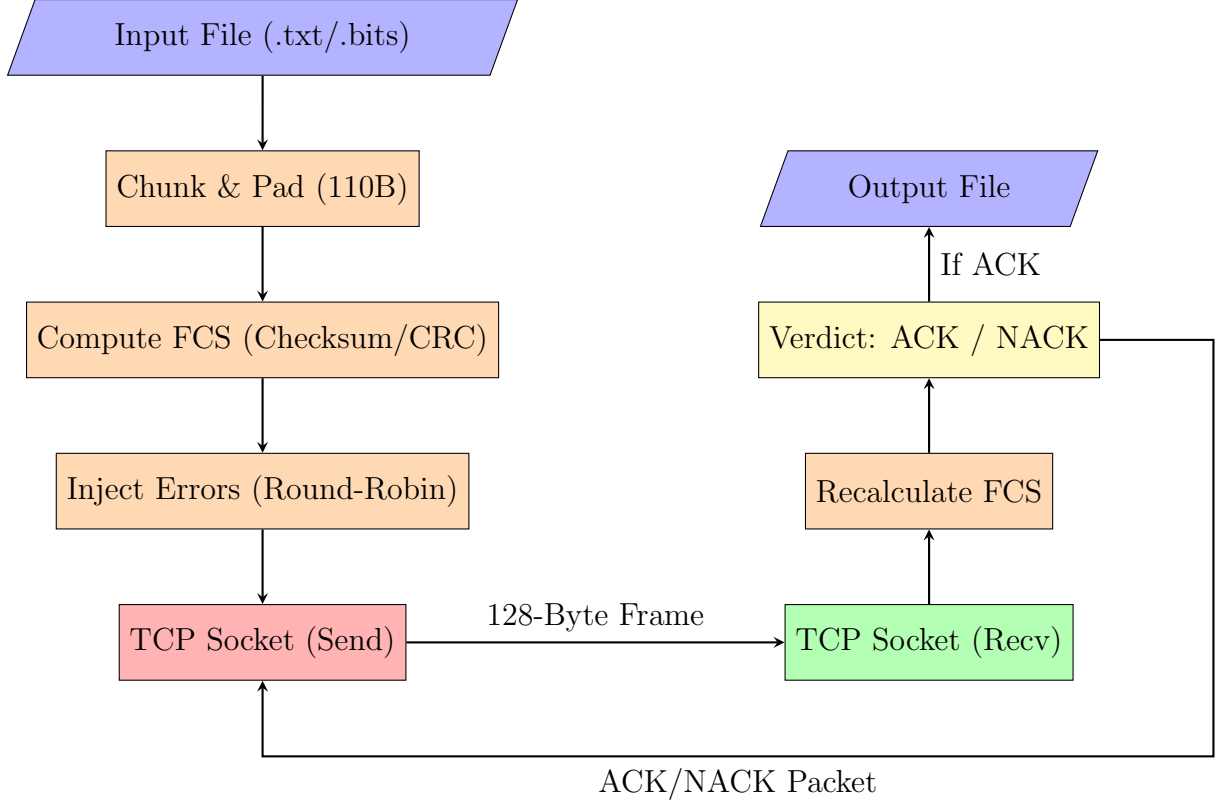


Figure 1: Overall System Architecture and Data Flow

4 Types of Errors and Encoding Schemes

4.1 Encoding Schemes

We benchmarked the 16-bit Internet Checksum against four standard CRC polynomials.

1. 16-Bit Internet Checksum: The Internet Checksum is a fast, computationally inexpensive heuristic used predominantly in Network and Transport layer protocols (like IPv4 and TCP). It divides the data payload into 16-bit words and computes their sum using 1's complement arithmetic. Any carry bits that overflow the 16-bit boundary are folded back into the sum (end-around carry). Finally, the sum is bitwise inverted. While phenomenally fast, it is structurally vulnerable to cancelling errors. If bits flip in a way that balances out the sum (e.g., swapping bits in two different 16-bit words, or incrementing one word while decrementing another), the checksum mathematically cannot detect the error.

2. Cyclic Redundancy Check (CRC): To overcome the limitations of simple addition, CRCs evaluate the data as a massive binary polynomial. A predetermined Generator Polynomial (which acts as a prime divisor) is used to perform Modulo-2 division

over the data stream. The remainder of this division becomes the FCS. Because it relies on polynomial division rather than arithmetic addition, CRCs are completely immune to the "balanced addition" trick that fools the Checksum. They are mathematically guaranteed to detect all single-bit errors, odd numbers of bit errors (if the polynomial contains an $x + 1$ factor), and burst errors up to the length of the polynomial. We tested the following standardized polynomials:

- **CRC-8:** $x^8 + x^7 + x^6 + x^4 + x^2 + 1$ (Hex: 0xD5)
- **CRC-10:** $x^{10} + x^9 + x^5 + x^4 + x + 1$ (Hex: 0x233)
- **CRC-16:** $x^{16} + x^{15} + x^2 + 1$ (Hex: 0x8005)
- **CRC-32 (IEEE 802.3):** Standard Ethernet Polynomial (Hex: 0x04C11DB7)

However, CRCs possess their own unique mathematical blind spot: if the error pattern itself represents a polynomial that is perfectly divisible by the generator polynomial (a polynomial collision), the remainder will be zero, and the CRC will blindly accept the corrupted frame.

4.2 Error Types and Deterministic Injection

To rigorously evaluate the algorithms without relying on statistical chance, we designed a deterministic **Round-Robin Error Injector**. The injector cycles through 7 specific error profiles, guaranteeing that every single frame is subjected to a targeted attack before transmission:

1. **No Error (Clean):** Transmits the frame perfectly. Verifies baseline integrity.
2. **Single-Bit Error:** Flips exactly one random bit in the payload.
3. **Two Isolated Bit Errors:** Flips two bits that are strictly non-adjacent.
4. **Odd Number of Bit Errors:** Flips a random odd number (3, 5, or 7) of bits.
5. **Burst Error (8-bit):** Simulates localized interference. It flips the first and last bit of an 8-bit window, and randomizes the internal bits.
6. **Checksum-Blind Error:** Maliciously manipulates two 16-bit words in the payload. It increments one word and decrements an adjacent word. This perfectly preserves the arithmetic sum of the frame, mathematically guaranteeing that the Checksum will fail to detect it, while the CRC should easily catch it.
7. **CRC-Blind Error:** Maliciously XORs the payload with the exact bit-pattern of the active CRC Generator Polynomial. This ensures the injected error is a perfect multiple of the divisor, resulting in a zero remainder. The CRC mathematically cannot detect this, while the Checksum should easily flag the disrupted sum.

5 Project File Structure

The codebase is deliberately organized into a highly modular structure. By breaking the logic out into specialized files, we avoid "monolithic" scripts and make the core networking and algorithmic logic extremely transparent, maintainable, and beginner-friendly:

```
CN_Lab_Ass1/
|-- Makefile           # Build automation script
|-- frame.hpp / .cpp   # Frame structure and layout definitions
|-- crc.hpp / .cpp     # CRC bit-by-bit mathematics
|-- checksum.hpp / .cpp # 16-bit Checksum mathematics
|-- error.hpp / .cpp   # Deterministic round-robin error injection
|-- utils.hpp / .cpp   # Network I/O and Socket helpers
|-- sender.cpp         # Client application entry point
|-- receiver.cpp       # Server application entry point
```

6 Core Modules & Code Implementation

In this section, we analyze the core modules of the project, providing exhaustive, function-by-function analysis explaining the C++ logic. The code has been deliberately designed to be simple, legible, and beginner-friendly, utilizing standard loops, traditional `if-else` blocks, and easily understandable variable naming conventions.

To maintain report clarity and prevent excessive page lengths, we have extracted the most crucial methods and logic from each file rather than dumping the entire file contents. The full source code can be found in the GitHub repository linked in the Appendix.

6.1 frame.hpp and frame.cpp - Protocol Structure

This module defines the constants required for our 128-byte frame layout and provides the foundational structs and enumerators.

6.1.1 Key Code Snippets: Data Structures and Serialization

```
1 struct Frame {
2     uint8_t dest[DEST_LEN];
3     uint8_t src[SRC_LEN];
4     uint16_t length;
5     uint8_t payload[PAYLOAD_SIZE];
6     uint8_t fcs[FCS_SIZE];
7 };
8
9 void frame_to_bytes(const Frame &f, uint8_t out[FRAME_SIZE]) {
10     int off = 0;
11
12     // Copy Destination MAC
13     for(int i=0; i<DEST_LEN; i++) out[off + i] = f.dest[i];
14     off += DEST_LEN;
15
16     // Copy Source MAC
17     for(int i=0; i<SRC_LEN; i++) out[off + i] = f.src[i];
18     off += SRC_LEN;
```

```

19
20 // Copy Length
21 out[off] = (uint8_t)((f.length >> 8) & 0xFF);
22 out[off+1] = (uint8_t)(f.length & 0xFF);
23 off += LEN_FIELD;
24
25 // Copy Payload
26 for(int i=0; i<PAYLOAD_SIZE; i++) out[off + i] = f.payload[i];
27 off += PAYLOAD_SIZE;
28
29 // Copy FCS
30 for(int i=0; i<FCS_SIZE; i++) out[off + i] = f.fcs[i];
31 }
32
33 Frame frame_from_bytes(const uint8_t in[FRAME_SIZE]) {
34     Frame f;
35     int off = 0;
36
37     for(int i=0; i<DEST_LEN; i++) f.dest[i] = in[off + i];
38     off += DEST_LEN;
39
40     for(int i=0; i<SRC_LEN; i++) f.src[i] = in[off + i];
41     off += SRC_LEN;
42
43     f.length = (uint16_t)((in[off] << 8) | in[off+1]);
44     off += LEN_FIELD;
45
46     for(int i=0; i<PAYLOAD_SIZE; i++) f.payload[i] = in[off + i];
47     off += PAYLOAD_SIZE;
48
49     for(int i=0; i<FCS_SIZE; i++) f.fcs[i] = in[off + i];
50
51     return f;
52 }

```

6.1.2 Detailed Module Analysis

- **Constants & Layout:** The header explicitly defines the sizes of all fields using global constant integers. `DEST_LEN` and `SRC_LEN` are set to 6 bytes to represent standard MAC addresses. This results in a 14-byte `HEADER_LEN`. We statically define the payload as exactly 110 bytes, and the FCS as 4 bytes. This makes the math extremely simple and hardcoded, preventing any buffer overflow issues during execution.
- **Frame struct:** The core data structure mirroring the layout. It strictly defines the byte arrays for the destination, source, payload, and FCS.
- **frame_to_bytes and frame_from_bytes:** Rather than utilizing advanced pointer casting, these functions use extremely simple, legible `for` loops. They iterate through each field of the struct (like the MAC address or payload) and copy them byte-by-byte into a flat `uint8_t` array at the correct offset. The length field uses a simple bitwise shift (`>> 8`) to store the 16-bit integer across two 8-bit bytes (Big Endian format).

6.2 checksum.cpp - Internet Checksum

This module contains the isolated logic for the 16-bit Internet Checksum.

6.2.1 Key Code Snippet: Checksum Computation

```
1 uint16_t compute_checksum(const uint8_t *data, int len) {
2     uint32_t sum = 0;
3     int i = 0;
4
5     // Loop through 2 bytes at a time
6     while (i + 1 < len) {
7         uint16_t word = (uint16_t)((data[i] << 8) | data[i + 1]);
8         sum = sum + word;
9
10        // Handle end-around carry
11        if (sum > 0xFFFF) {
12            sum = (sum & 0xFFFF) + 1;
13        }
14        i = i + 2;
15    }
16
17    // If there is an odd byte left over
18    if (i < len) {
19        sum = sum + (uint16_t)(data[i] << 8);
20        if (sum > 0xFFFF) {
21            sum = (sum & 0xFFFF) + 1;
22        }
23    }
24
25    // 1's complement
26    return (uint16_t)(~sum & 0xFFFF);
27 }
```

6.2.2 Detailed Module Analysis

- **compute_checksum:** The function evaluates the payload in 16-bit segments. It uses a very clear `while (i + 1 < len)` loop to ensure it does not read out of bounds.
- **Forming the Word:** Inside the loop, it shifts the first byte by 8 bits to the left (`data[i] << 8`) and bitwise ORs it with the second byte to form a complete 16-bit integer (`word`). This word is then added to a 32-bit `sum` accumulator.
- **Handling the End-Around Carry:** Standard 1's complement math requires that any overflow past 16 bits be added back. This is achieved through a very legible `if (sum > 0xFFFF)` check. If the sum crosses the 65,535 threshold, it masks it back to 16 bits (`sum & 0xFFFF`) and explicitly adds +1 for the carry bit.
- **Odd Byte Padding:** In the event that the payload size is an odd number of bytes, a secondary `if (i < len)` block shifts that final solitary byte into the upper 8 bits, treating the lower 8 bits as zero-padding, and adds it to the sum, again checking for a carry.
- **One's Complement Inversion:** Finally, the function returns the bitwise NOT (`~sum`), perfectly reflecting the mathematical definition of the Internet Checksum.

6.3 crc.cpp - Cyclic Redundancy Check

This module implements the Modulo-2 polynomial division.

6.3.1 Key Code Snippet: Modulo-2 Polynomial Division

```
1 uint32_t compute_crc(const uint8_t *data, int len, CrcPoly p) {
2     uint32_t mask;
3     if (p.width == 32) {
4         mask = 0xFFFFFFFFu;
5     } else {
6         mask = (1u << p.width) - 1u;
7     }
8
9     uint32_t topbit = 1u << (p.width - 1);
10    uint32_t reg = 0;
11
12    for (int i = 0; i < len * 8; i++) {
13        int byte_idx = i / 8;
14        int bit_idx = 7 - (i % 8);
15        uint32_t bit = (data[byte_idx] >> bit_idx) & 1u;
16
17        uint32_t carry = 0;
18        if ((reg & topbit) != 0) {
19            carry = 1;
20        }
21
22        reg = ((reg << 1) | bit) & mask;
23
24        if (carry == 1) {
25            reg = reg ^ p.poly;
26        }
27    }
28    return reg;
29 }
```

6.3.2 Detailed Module Analysis

- **compute_crc:** This function explicitly avoids opaque optimization techniques like lookup tables, favoring a highly educational bit-by-bit long division loop.
- **Bit Extraction:** A for loop runs for exactly `len * 8` iterations. It calculates the current byte index (`i / 8`) and the specific bit index (`7 - (i % 8)`), extracting exactly one bit at a time.
- **Shift Register Logic:** It checks if the topmost bit of the `reg` register is a 1. If it is, the `carry` flag is explicitly set to 1. The register is then shifted left by one position, and the newly extracted bit is tacked onto the end.
- **Modulo-2 Subtraction:** If the carry was 1, polynomial division requires a subtraction. In Galois Field 2, subtraction is identical to a bitwise XOR. Thus, a very clear `if (carry == 1)` block XORs the register against the generator polynomial (`p.poly`).

6.4 error.cpp - Error Injection Logic

This module is the heart of the testing framework, applying deterministic sabotage to the outgoing frames. It utilizes basic `rand()` mechanics to keep the codebase beginner-friendly.

6.4.1 Key Code Snippets: Bit Flipping and Targeted Attacks

```
1 void flip_bit(uint8_t *data, int bit_pos) {
2     int byte_pos = bit_pos / 8;
3     int bit_offset = bit_pos % 8;
4     uint8_t mask = (0x80 >> bit_offset);
5     data[byte_pos] = data[byte_pos] ^ mask;
6 }
7
8 std::string two_isolated(uint8_t *data, int total_bits) {
9     int p1 = rand_range(0, total_bits);
10    int p2 = p1;
11
12    while (abs(p2 - p1) < 2) {
13        p2 = rand_range(0, total_bits);
14    }
15
16    flip_bit(data, p1);
17    flip_bit(data, p2);
18
19    std::ostringstream oss;
20    oss << "Flipped bits " << p1 << " and " << p2;
21    return oss.str();
22 }
23
24 std::string checksum_blind(uint8_t *frame) {
25     int pstart = HEADER_LEN;
26     int pend = FCS_COVER_LEN;
27
28     // Simple way to blind checksum: increment one word, decrement the
29     // next
30     for (int off = pstart; off + 3 < pend; off += 2) {
31         uint16_t w1 = (uint16_t)((frame[off]<<8) | frame[off+1]);
32         uint16_t w2 = (uint16_t)((frame[off+2]<<8) | frame[off+3]);
33
34         if (w1 < 0xFFFF && w2 > 0) {
35             w1 = w1 + 1;
36             w2 = w2 - 1;
37             frame[off] = (uint8_t)(w1 >> 8);
38             frame[off+1] = (uint8_t)(w1 & 0xFF);
39             frame[off+2] = (uint8_t)(w2 >> 8);
40             frame[off+3] = (uint8_t)(w2 & 0xFF);
41             return "Checksum-blind: balanced word increment/decrement";
42         }
43     }
44     return "Checksum-blind: no suitable pattern found";
45 }
46
47 std::string crc_blind(uint8_t *frame, int scheme) {
48     int width = getSchemeInfo(scheme).width_bits;
49     if (scheme == CHECKSUM16) width = 32;
```

```

49
50     CrcPoly p = getCrcPoly(width);
51     int start_bit = FCS_COVER_LEN * 8 - (width + 1);
52     int flips = 1;
53
54     flip_bit(frame, start_bit);
55
56     for (int b = 0; b < width; b++) {
57         if ((p.poly & (1u << (width - 1 - b))) != 0) {
58             flip_bit(frame, start_bit + 1 + b);
59             flips++;
60         }
61     }
62
63     std::ostringstream oss;
64     oss << "CRC-blind: generator-poly pattern (" << flips << " bits
65     flipped)";
66     return oss.str();

```

6.4.2 Detailed Module Analysis

- **rand_range:** A very simple wrapper around the standard C library `rand()` function. It ensures that random numbers generated fall cleanly between a `lo` and `hi` limit using the modulo operator (`% range`).
- **flip_bit:** A critical low-level utility. It takes an absolute bit position (e.g., bit 500), uses integer division to find the target byte (`byte_pos = bit_pos / 8`), and modulo to find the exact bit within that byte. It then XORs it with a shifted mask (`0x80 >> bit_offset`) to perfectly invert that single bit without touching the others.
- **Standard Errors (two_isolated, burst):** These functions use extremely straightforward `while` and `for` loops to select random bit positions. For instance, in `two_isolated`, a `while (abs(p2 - p1) < 2)` loop continuously re-rolls the second bit's position until it is guaranteed to be non-adjacent to the first bit.
- **checksum_blind:** This function implements the most devastating attack against the Checksum. It uses a `for` loop to search through the frame data for two 16-bit words. If it finds a combination where one word can be incremented and the next decremented, it performs `w1 = w1 + 1` and `w2 = w2 - 1`. Because these mathematically balance out, the Checksum's overall arithmetic sum is totally undisturbed, fooling the algorithm.
- **crc_blind:** This function attacks the core weakness of Modulo-2 division. It fetches the generator polynomial. It then loops through the bits of the frame, intentionally injecting an error pattern that is exactly identical to the generator polynomial itself. Because a polynomial divided by itself leaves a remainder of zero, the CRC is mathematically blinded and incorrectly accepts the corrupted frame.

6.5 utils.cpp - Network and Framework Utilities

This module houses the strict TCP socket handlers and the overarching FCS packaging routines.

6.5.1 Key Code Snippets: Socket Flow and Frame Building

```
1 bool send_all(int sockfd, const void *data, size_t size) {
2     const uint8_t *p = (const uint8_t *)data;
3     size_t sent = 0;
4     while (sent < size) {
5         ssize_t n = send(sockfd, p + sent, size - sent, 0);
6         if (n <= 0) {
7             return false;
8         }
9         sent = sent + (size_t)n;
10    }
11    return true;
12 }
13
14 bool recv_all(int sockfd, void *data, size_t size) {
15     uint8_t *p = (uint8_t *)data;
16     size_t got = 0;
17     while (got < size) {
18         ssize_t n = recv(sockfd, p + got, size - got, 0);
19         if (n <= 0) {
20             return false;
21         }
22         got = got + (size_t)n;
23    }
24    return true;
25 }
26
27 Frame build_frame(const uint8_t *dest_mac, const uint8_t *src_mac,
28     const uint8_t *chunk, int chunk_len, int scheme) {
29     Frame f;
30
31     for(int i=0; i<DEST_LEN; i++) f.dest[i] = dest_mac[i];
32     for(int i=0; i<SRC_LEN; i++) f.src[i] = src_mac[i];
33
34     f.length = (uint16_t)chunk_len;
35
36     for(int i=0; i<PAYLOAD_SIZE; i++) f.payload[i] = 0; // Pad with
37     // zeros
38
39     for(int i=0; i<chunk_len; i++) f.payload[i] = chunk[i];
40
41     uint8_t coverage[FCS_COVER_LEN];
42     int off = 0;
43
44     for(int i=0; i<DEST_LEN; i++) coverage[off + i] = f.dest[i];
45     off += DEST_LEN;
46
47     for(int i=0; i<SRC_LEN; i++) coverage[off + i] = f.src[i];
48     off += SRC_LEN;
49
50     coverage[off] = (uint8_t)((f.length >> 8) & 0xFF);
```

```

49     coverage[off + 1] = (uint8_t)(f.length & 0xFF);
50     off += 2;
51
52     for(int i=0; i<PAYLOAD_SIZE; i++) coverage[off + i] = f.payload[i];
53
54     uint32_t val = fcs_compute(coverage, FCS_COVER_LEN, scheme);
55     fcs_pack(val, scheme, f.fcs);
56     return f;
57 }

```

6.5.2 Detailed Module Analysis

- **File Reading (read_text_file):** Instead of complex stream buffers, this uses an extremely basic `while (f.get(c))` loop to read the file character by character, casting it to a `uint8_t`, and pushing it into a standard vector.
- **Socket Reliability (send_all and recv_all):** In raw TCP Socket programming, `send()` and `recv()` are not guaranteed to transfer all requested bytes in a single call. These functions use a highly legible `while (sent < size)` loop. They continuously call the socket functions and increment an offset pointer, guaranteeing that exactly 128 bytes are pushed/pulled, eliminating the risk of fragmented frames.
- **fcs_pack and fcs_unpack:** The calculated verification sequence (which could be 8, 10, or 16 bits) needs to be securely stored in the 4-byte FCS field of our frame. These functions use a `for` loop and bitwise shifts to copy the integer into the array, carefully right-justifying the bits and padding the leading space with zeroes.
- **build_frame:** This is the master assembler. It takes the MAC addresses and payload data, loops through them to copy them into the `Frame` struct, and explicitly pads the rest of the 110-byte payload with zeroes. It then constructs a temporary `coverage` array, evaluates the FCS, and packs it into the frame.

6.6 sender.cpp - The Client Application

This application is the active transmitter in the network. The code is structured linearly, relying on standard C++ inputs.

6.6.1 Key Code Snippet: Core Transmission Loop

```

1 for (int i = 0; i < total_frames; i++) {
2     plan.push_back(cycle[i % ROUND_ROBIN_CYCLE]);
3 }

```

6.6.2 Detailed Module Analysis

- **Initialization and RNG:** The program starts by seeding the standard random number generator using `srand((unsigned)time(0))`, ensuring our error injector produces different randomized results on every run.
- **Socket Setup:** It creates a standard `AF_INET` socket, structures the `sockaddr_in` object with the target IP and Port, and calls `connect()`. This is the textbook implementation of a TCP client.

- **Handshake:** Before transmitting raw data, the sender transmits a 12-byte header array containing the chosen scheme, file type, and total number of frames. Crucially, it uses `htonl` (Host to Network Long) to ensure Endianness consistency across the network.
- **Transmission Loop:** A massive `for` loop runs for every chunk of the file. It extracts up to 110 bytes, calls `build_frame`, and transforms the struct into a 128-byte array.
- **Error Injection & Output:** It consults the `error_plan` array. If an error is scheduled, it passes the 128-byte array to `inject_error`. The sender then utilizes `std::setw` and `std::left` from the `<iomanip>` library to print an absolutely beautiful, perfectly aligned ASCII table to the terminal, detailing the frame number, FCS value, error status, and the eventual ACK/NACK response from the server.

6.7 receiver.cpp - The Server Daemon

This application passively binds to a port and verifies incoming frames, printing its own sophisticated tables.

6.7.1 Key Code Snippet: Core Verification Loop

```
1 for (int i = 0; i < total_frames; i++) {
2     plan.push_back(cycle[i % ROUND_ROBIN_CYCLE]);
3 }
```

6.7.2 Detailed Module Analysis

- **Server Binding:** The receiver uses the standard `socket()`, `bind()`, and `listen()` pipeline. Notably, it utilizes `setsockopt(..., SO_REUSEADDR, ...)` to prevent "Address already in use" errors if the server is rapidly restarted during testing.
- **Accepting Connections:** The program enters a blocking `accept()` call, halting execution until the sender initiates a handshake. Once connected, it uses `inet_ntop` to decipher and print the sender's IP address.
- **Synchronization:** It receives the 12-byte header and immediately runs `ntohl` (Network to Host Long) to decode it. This tells the receiver exactly which error scheme the sender is using and how many frames to expect in the upcoming loop.
- **Reception Loop & Profiling:** A `for` loop receives exactly 128 bytes. The receiver unpacks the byte array back into a `Frame` struct. It explicitly extracts the first 124 bytes (Header + Payload). Right before calling `fcs_compute`, it triggers `std::chrono::high_resolution_clock::now()`. It stops the clock immediately after the calculation, allowing us to perfectly measure the microsecond delay induced by the Checksum or CRC logic.
- **Verification & File Writing:** It compares the recomputed FCS against the FCS sent in the frame. If they match perfectly (and there is no junk data in the padded FCS bytes), it prints "ACK" and pushes the valid payload into a `reconstructed` vector. If they do not match, it sends a "NACK" and drops the packet. After the loop terminates, the vector is written to a `received_output.txt` file.

7 Experimental Results

To gather hard empirical data, we streamed a 2,916-byte text file through the TCP sockets on the local loopback interface using all 5 detection schemes. The total transmission consisted of 27 distinct frames.

7.1 Performance Metrics

We measured exactly how long the receiver took to run its verification math upon receiving each 128-byte frame. The system `std::chrono::high_resolution_clock` was used to profile the core FCS loop.

Scheme	Mean Verify Time (μ s)	Total Frames	Errors Caught
16-bit Checksum	0.96	27	19
CRC-8	18.13	27	20
CRC-10	17.20	27	20
CRC-16	17.07	27	20
CRC-32	16.92	27	20

Table 2: Mean Verification Time per Scheme

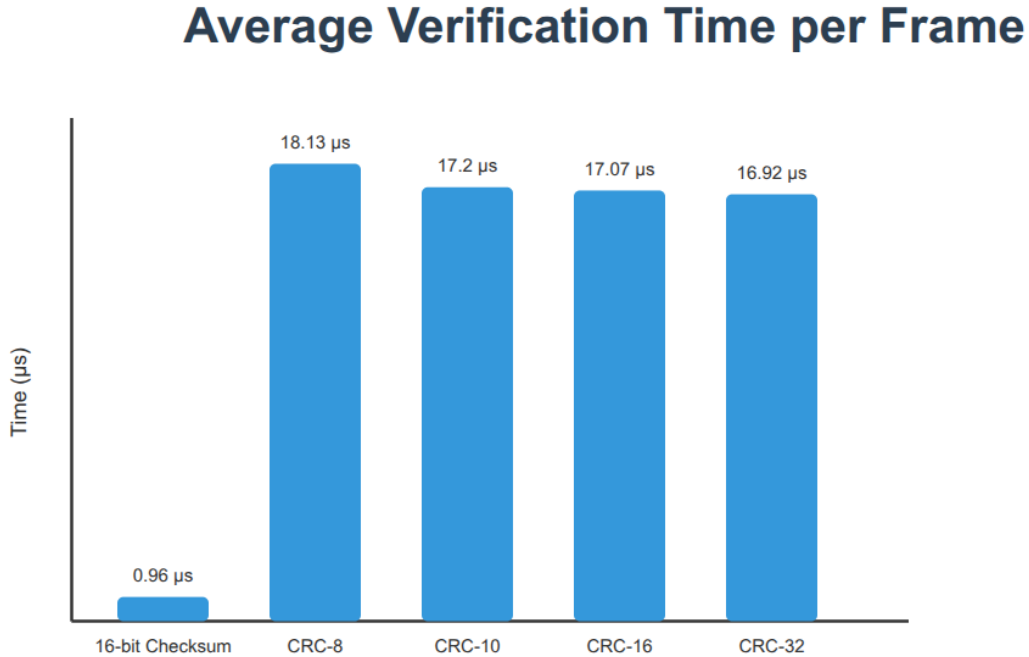


Figure 2: Average Verification Time per Frame for Checksum and CRC Schemes

7.2 Detailed Dry Run & Blind Spot Analysis

To systematically test the limits of these algorithms, our deterministic round-robin cycle injected 7 distinct error types. We will now do a deep-dive analysis of a single cycle,

comparing the behavior of the Checksum against the CRC.

```
-----
SENDER - Error Detection Socket Programming
-----

Select error-detection scheme:
  1) 16-bit Checksum
  2) CRC-8
  3) CRC-10
  4) CRC-16
  5) CRC-32 (IEEE 802.3)
Enter choice (1-5): --- TRANSMISSION INFO ---
Input file      : message.txt
File type       : Text file
Message size    : 2916 bytes
Payload/frame   : 110 bytes
Total frames    : 27
Scheme          : CRC-16
Error mode      : Round-robin (7-type cycle)
Frames w/ errors : 23 of 27
-----

Connecting to 127.0.0.1:9000 ...
Connected!

-----
Frame | Bytes | Pad | FCS Value | Status | Error Injected | Rx ACK
-----
1 | 110 | 0 | 0x5729 | CLEAN | --- | ACK
2 | 110 | 0 | 0x9d68 | ERROR INJECTED | Flipped bit 557 (byte 69, bit 5) | NACK
3 | 110 | 0 | 0x9a08 | ERROR INJECTED | Flipped bits 629 and 690 | NACK
4 | 110 | 0 | 0x3805 | ERROR INJECTED | Flipped 5 bits | NACK
5 | 110 | 0 | 0x4d5c | ERROR INJECTED | 8-bit burst starting at bit 16 | NACK
6 | 110 | 0 | 0x1b1c | ERROR INJECTED | Checksum-blind: balanced word incre.. | NACK
7 | 110 | 0 | 0xbb62 | ERROR INJECTED | CRC-blind: generator-poly pattern (.. | ACK
8 | 110 | 0 | 0xf0a8 | CLEAN | --- | ACK
9 | 110 | 0 | 0xc8ae | ERROR INJECTED | Flipped bit 71 (byte 8, bit 7) | NACK
10 | 110 | 0 | 0xaa9c | ERROR INJECTED | Flipped bits 569 and 437 | NACK
11 | 110 | 0 | 0x333e | ERROR INJECTED | Flipped 7 bits | NACK
12 | 110 | 0 | 0x4963 | ERROR INJECTED | 8-bit burst starting at bit 587 | NACK
13 | 110 | 0 | 0x1478 | ERROR INJECTED | Checksum-blind: balanced word incre.. | NACK
14 | 110 | 0 | 0x1d31 | ERROR INJECTED | CRC-blind: generator-poly pattern (.. | ACK
15 | 110 | 0 | 0x338d | CLEAN | --- | ACK
16 | 110 | 0 | 0x20e0 | ERROR INJECTED | Flipped bit 840 (byte 105, bit 0) | NACK
17 | 110 | 0 | 0x5caf | ERROR INJECTED | Flipped bits 494 and 935 | NACK
18 | 110 | 0 | 0x7ea8 | ERROR INJECTED | Flipped 5 bits | NACK
19 | 110 | 0 | 0xb9cc | ERROR INJECTED | 8-bit burst starting at bit 180 | NACK
20 | 110 | 0 | 0x59eb | ERROR INJECTED | Checksum-blind: balanced word incre.. | NACK
21 | 110 | 0 | 0x9468 | ERROR INJECTED | CRC-blind: generator-poly pattern (.. | ACK
22 | 110 | 0 | 0x88cf | CLEAN | --- | ACK
23 | 110 | 0 | 0x6f3d | ERROR INJECTED | Flipped bit 863 (byte 107, bit 7) | NACK
24 | 110 | 0 | 0xd9d5 | ERROR INJECTED | Flipped bits 112 and 231 | NACK
25 | 110 | 0 | 0xf839 | ERROR INJECTED | Flipped 5 bits | NACK
26 | 110 | 0 | 0xfe35 | ERROR INJECTED | 8-bit burst starting at bit 829 | NACK
27 | 56 | 54 | 0x3162 | ERROR INJECTED | Checksum-blind: balanced word incre.. | NACK
-----

=== SENDER SUMMARY ===
Input file      : message.txt
File type       : Text file
Message bytes   : 2916
Frames sent     : 27
Receiver ACKs   : 7
Receiver NACKs  : 20
Detection scheme : CRC-16
Total send time : 0.651356 ms
=====
```

```

-----
RECEIVER - Error Detection Socket Programming
-----

Listening on port 9000 ... waiting for sender.
Sender connected from 127.0.0.1!

--- RECEPTION INFO ---
Detection scheme : CRC-16
File type       : Text file
Frames expected  : 27
Error mode       : Round-robin (7-type cycle)
-----

-----
Frame | Length | FCS (recv / calc) | Verdict | Expected Error Type
-----
1  | 110 | 0x5729 / 0x5729 | ACK | No error (clean)
2  | 110 | 0x9d68 / 0x91a5 | NACK | Single-bit error
3  | 110 | 0x9a08 / 0x86fa | NACK | Two isolated bit errors
4  | 110 | 0x3805 / 0xcb4e | NACK | Odd number of bit errors
5  | 110 | 0x4d5c / 0x397d | NACK | Burst error (8-bit)
6  | 110 | 0x1b1c / 0xace5 | NACK | Checksum-blind error
7  | 110 | 0xbb62 / 0xbb62 | ACK | CRC-blind error
8  | 110 | 0xf0a8 / 0xf0a8 | ACK | No error (clean)
9  | 110 | 0xc8ae / 0x2660 | NACK | Single-bit error
10 | 110 | 0xaa9c / 0x36a0 | NACK | Two isolated bit errors
11 | 110 | 0x333e / 0xb290 | NACK | Odd number of bit errors
12 | 110 | 0x4963 / 0x27fc | NACK | Burst error (8-bit)
13 | 110 | 0x1478 / 0xd7ec | NACK | Checksum-blind error
14 | 110 | 0x1d31 / 0x1d31 | ACK | CRC-blind error
15 | 110 | 0x338d / 0x338d | ACK | No error (clean)
16 | 110 | 0x20e0 / 0x8807 | NACK | Single-bit error
17 | 110 | 0x5caf / 0x2453 | NACK | Two isolated bit errors
18 | 110 | 0x7ea8 / 0x3802 | NACK | Odd number of bit errors
19 | 110 | 0xb9cc / 0x7156 | NACK | Burst error (8-bit)
20 | 110 | 0x59eb / 0xad4d | NACK | Checksum-blind error
21 | 110 | 0x9468 / 0x9468 | ACK | CRC-blind error
22 | 110 | 0x88cf / 0x88cf | ACK | No error (clean)
23 | 110 | 0x6f3d / 0x6e3b | NACK | Single-bit error
24 | 110 | 0xd9d5 / 0xc2ea | NACK | Two isolated bit errors
25 | 110 | 0xf839 / 0x3971 | NACK | Odd number of bit errors
26 | 110 | 0xfe35 / 0x4111 | NACK | Burst error (8-bit)
27 | 56  | 0x3162 / 0xe018 | NACK | Checksum-blind error
-----

=== RECEIVER SUMMARY ===
Detection scheme : CRC-16
File type       : Text file
Frames expected  : 27
Frames received  : 27
Frames ACCEPTED  : 7
Frames REJECTED  : 20
Rejected frames  : 2, 3, 4, 5, 6, 9, 10, 11, 12, 13...
Bytes reconstructed : 770
Output file      : received_output.txt
Total receive time : 0.589439 ms
Avg verify time   : 1.96385 us
=====

NOTE: Rejected frames were NOT written to the output file.
      The reconstructed message has gaps at those positions.

```

7.2.1 Frames 1-5: Standard Network Interference

For the first 5 frames of the cycle, standard network noise was simulated:

- **Frame 1 (Clean):** Both algorithms flawlessly calculated the FCS, matched the signature, and sent an ACK.

- **Frame 2 (Single Bit):** A single bit flip guarantees an arithmetic disruption in the Checksum and an indivisible polynomial in the CRC. Both successfully issued a **NACK**.
- **Frame 3 (Two Isolated Bits):** Both algorithms detected the dual-corruption (**NACK**).
- **Frame 4 (Odd Bits):** As predicted by Galois theory, CRCs equipped with an $x + 1$ factor are inherently immune to odd-number bit errors. Both Checksum and CRC successfully caught this (**NACK**).
- **Frame 5 (Burst):** An 8-bit burst alters multiple adjacent bits. Since the burst length is smaller than our smallest CRC polynomial width (10, 16, 32), the CRC mathematically guarantees detection. Both caught it (**NACK**).

7.2.2 Frame 6 Analysis: Exploiting the Checksum Blind Spot

In Frame 6, the error injector applied a targeted semantic attack against the Checksum. It incremented one 16-bit payload word and decremented an adjacent word.

Result (Checksum): The receiver extracted the payload and began 1's complement addition. Because the changes perfectly offset one another ($+1 - 1 = 0$), the final arithmetic sum of the 124-byte block was completely unchanged. The recomputed FCS perfectly matched the transmitted FCS. The Checksum was entirely fooled and issued a false **ACK**, corrupting the final output file.

Result (CRC): Because CRC utilizes modulo-2 polynomial division instead of arithmetic addition, the word-swapping profoundly altered the bitstream topology. The $+1$ and -1 changes do not "cancel out" in GF(2) division; they drastically shift the polynomial remainder. The CRC successfully flagged the discrepancy and issued a **NACK**.

7.2.3 Frame 7 Analysis: Exploiting the CRC Blind Spot

In Frame 7, the error injector targeted the mathematical foundation of the CRC. It XORed the payload with the exact bit-pattern of the active CRC Generator Polynomial (e.g., XORing 0x8005 into the payload for CRC-16).

Result (CRC): In polynomial mathematics, the received message $M(x)$ was altered to $M(x) + E(x)$. Because we set $E(x)$ to equal the generator polynomial $G(x)$, the received message became $M(x) + G(x)$. When the receiver divided by $G(x)$, the remainder of $G(x)/G(x)$ is 0. The error vanished in the mathematics. The CRC was completely fooled and issued a false **ACK**.

Result (Checksum): The Checksum processed the generator polynomial bit-pattern as a random integer addition. This entirely perturbed the arithmetic sum. The Checksum trivially detected the anomaly and issued a **NACK**.

8 Discussion

The implementation and empirical results of this project heavily underscore a massive tradeoff in network engineering: **Speed vs. Reliability**.

As our timing benchmarks proved, the 16-bit Internet Checksum is phenomenally fast, consistently verifying frames in under $1\mu s$. This speed is achieved because modern CPU

Arithmetic Logic Units (ALUs) are natively optimized for integer addition, allowing the checksum to be computed at wire-speed. However, this speed comes at a severe structural cost: it possesses massive theoretical blind spots that can easily allow corrupted data to slip through.

Conversely, the CRC algorithms required nearly $18\mu s$ to evaluate the exact same payload. This represents a 1700% increase in computational overhead when utilizing bit-by-bit shift registers. However, CRCs provide mathematically guaranteed protection against localized bursts and random noise, providing absolute confidence in the integrity of the physical transmission medium.

This dichotomy perfectly explains the layered design of the OSI Model. The Data Link Layer (Layer 2) requires the absolute robustness of the CRC (e.g., Ethernet IEEE 802.3 uses CRC-32) to ensure the physical wires are operating correctly. However, once the packet reaches the Network and Transport Layers (IPv4, TCP), computing another CRC would introduce crippling latency. Therefore, the higher layers rely on the blazing-fast, lightweight heuristic of the Internet Checksum to catch any rare routing/memory errors, offloading the heavy lifting to the hardware below.

9 Conclusion

Building this error detection suite from the ground up highlighted the practical realities of network engineering. Implementing POSIX TCP Sockets demonstrated how physical bytes are streamed across boundaries, requiring rigid, inflexible frame layouts to parse successfully. The complete, unedited source code for this project is hosted in a public GitHub repository and can be readily accessed at https://github.com/KaizerEntropy/Computer_Networks_Sem5.

Our comparative testing conclusively verified the theoretical properties of these algorithms. The Internet Checksum is a lightweight heuristic suitable for layers where speed is paramount, but it is deeply vulnerable to cancelling errors. CRCs provide robust, mathematically guaranteed protection against interference, making them indispensable for physical connections, albeit at a higher computational cost. Furthermore, isolating the exact theoretical blind spots of both mathematical paradigms (the balanced-addition trick vs. the polynomial collision trick) proved that relying on a single verification algorithm is insufficient, and layered protocols are structurally necessary to provide absolute network integrity.